NexoEventos · Lista de endpoints

Ingeniería de Datos · Universidad del Rosario · Entregable 1

5. Endpoints

Todas las rutas aceptan el encabezado `X-Usuario`. Códigos: **201** al crear, **204** al borrar, **404** si no existe, **409** si una regla de la base lo rechaza o hay duplicados, **422** si los datos son inválidos, **503** si la base no responde.

Método	Ruta	Qué hace
GET, POST	/clientes/	Listar o buscar (`q`, `tipo`) y registrar
GET	/clientes/{id}	Consultar cliente
GET	/clientes/{id}/eventos	Eventos de un cliente
GET, POST	/staff/	Listar (`area`, `q`) y registrar con jefe
GET	/staff/{id}	Consultar empleado
GET	/salones/	Catálogo de salones
GET	/salones/disponibles?inicio=&fin=&aforo=&excluir_evento=	Salones disponibles (A5)
PATCH	/salones/{id}	Habilitar o deshabilitar
GET	/servicios/	Catálogo de servicios
PATCH	/servicios/{id}	Cambiar precio o disponibilidad
GET, POST	/eventos/	Listar con filtros (`estado`, `cliente_id`, `salon_id`, `desde`, `hasta`, `q`) y crear
GET	/eventos/{id}	Detalle con servicios y resumen de asistencia
PATCH	/eventos/{id}	Reprogramar: salón, horario, aforo, coordinador
PATCH	/eventos/{id}/estado	Confirmar, finalizar o cancelar
GET, POST	/eventos/{id}/servicios	Ver y agregar servicios
PUT, DELETE	/eventos/{id}/servicios/{servicio_id}	Cambiar cantidad o quitar servicio
GET	/eventos/{id}/cotizacion	Desglose salón + servicios = total
GET	/eventos/{id}/auditoria	Historial de cambios
GET, POST	/eventos/{id}/inscripciones	Inscritos (`llegaron=true/false`) e inscribir por documento
POST	/eventos/{id}/checkin	Registrar llegada por documento
GET, POST	/asistentes/	Listar o buscar y registrar
GET	/asistentes/{id}	Consultar asistente
GET	/reportes/organigrama?staff_id=	B1
GET	/reportes/escalamiento/{staff_id}	B2
GET	/reportes/ranking-clientes	B3
GET	/reportes/ocupacion-salones?salon_id=	B4

Método	Ruta	Qué hace
GET	/reportes/tablero	Datos del tablero de inicio

Ejemplos de petición y respuesta

Salones disponibles (A5)

```
GET /salones/disponibles?inicio=2026-11-18T09:00&fin=2026-11-
18T11:00&aforo=50
200 [
  {"salon_id": 2, "nombre": "Salón Heliconia", "tamano": "Pequeño",
"capacidad": 60, "precio_hora": 320000.0, "costo_estimado": 640000.0},
  {"salon_id": 4, "nombre": "Salón Ceiba", "tamano": "Mediano",
"capacidad": 200, "precio_hora": 800000.0, "costo_estimado": 1600000.0},
  {"salon_id": 5, "nombre": "Gran Salón Cóndor", "tamano": "Grande",
"capacidad": 600, "precio_hora": 1800000.0, "costo_estimado": 3600000.0}
]
```

Crear evento que se cruza (A1)

```
POST /eventos/
{"cliente_id": 2, "salon_id": 3, "nombre": "Taller de Economía Circular",
"tipo": "Seminario",
 "inicio": "2026-11-18T12:30", "fin": "2026-11-18T14:00",
"aforo_esperado": 50}

409 {"detail": "DISPONIBILIDAD: El salón \"Salón Guayacán\" está
reservado para \"Jornada Científica Solvida\" (2026-11-18 08:00 a 2026-
11-18 12:00). Se requiere 1 hora de montaje entre eventos.",
     "regla": "DISPONIBILIDAD", "codigo_sql": "P0001", "sugerencia":
"Consulte los salones disponibles para ese horario."}
```

Crear evento válido

```
POST /eventos/   (mismo cuerpo, "inicio": "2026-11-18T13:00", "fin":
"2026-11-18T15:00")
201 {"id": 16, "estado": "Cotizado", "valor_total": 1300000.0,
"tasa_asistencia": null, ...}
```

Agregar servicio sin cantidad (A3)

```
POST /eventos/16/servicios   {"servicio_id": 2}
201 {"servicio": "Estación de café", "unidad_cobro": "Por persona",
"cantidad": 50, "cantidad_automatica": true,
     "precio_unitario": 6500.0, "subtotal": 325000.0,
"valor_total_evento": 1625000.0}
```

Cupo lleno y check-in fuera de horario (A2)

```
POST /eventos/8/inscripciones   {"documento": "1000100001"}
409 {"detail": "CUPO: El evento \"Club de Lectura Páramo\" alcanzó su
aforo contratado (20 personas).", "regla": "CUPO", ...}

POST /eventos/7/checkin   {"documento": "1000100001", "fecha_hora":
```

```
"2026-11-18T05:00"}
409 {"detail": "CHECK-IN: Solo se permite ingresar entre 2026-11-18 07:00
y 2026-11-18 12:00 (hora indicada: 2026-11-18 05:00).", "regla": "CHECK-
IN", ...}
```

Finalizar (A4)

```
PATCH /eventos/6/estado    X-Usuario: farango    {"estado": "Finalizado"}
200 {"id": 6, "nombre": "Feria Empresarial del Caribe", "estado":
"Finalizado", "valor_total": 28050000.0, "tasa_asistencia": 75.0, ...}

GET /eventos/6/auditoria   (última fila)
{"campo": "estado", "valor_anterior": "Confirmado", "valor_nuevo":
"Finalizado", "usuario": "farango", "fecha": "..."}
```

Datos inválidos (DTO)

```
POST /eventos/   (fin anterior al inicio)
422 {"detail": "La fecha/hora de fin debe ser posterior a la de inicio",
"regla": "VALIDACION", "errores": [...]}
```

6. Integración API ↔ base

`db.commit()` dispara los triggers; inmediatamente después `db.refresh(obj)` trae los campos que calculó la base (`valor_total`, `tasa_asistencia`). Si un trigger lanza `RAISE EXCEPTION`, SQLAlchemy la entrega como `DBAPIError`; `app/errores.py` lee `exc.orig.pgcode` y `exc.orig.diag` (mensaje, hint, restricción) y arma la respuesta sin trazas de Python. Todo el SQL manual usa `text()` con parámetros (`:id`), nunca concatenación, y la cadena de conexión vive en `.env`.